

Now with orchestrator picker

ONE AGENT TO ROUTE THEM ALL

The installer now starts with prompt 0 that lets you pick which AI agent orchestrator to install into. Claude Code is the default, but you can also pick Cursor, Cline, Codex CLI, Aider, or generate a portable `SYSTEM_PROMPT.md` for any other tool. This capture shows the Claude-Code path; the flow is identical for the others, just with a different output file.

What is new here: a prompt 0. Which agent orchestrator do you use? at the top of the wizard. The installer then writes the appropriate rules file (or hooks) for that orchestrator, and for non-Claude tools it also copies the shipped skills so the model can read them.

1. The orchestrator choice

c	claude-code (default)	writes <code>~/.claude/settings.json</code> hooks + symlinks skills into <code>~/.claude/skills/</code>
r	cursor	writes <code>./.cursor/rules/sauron.mdc</code> with <code>alwaysApply:true</code> ; copies skills alongside
l	cline	writes <code>./.clinerules</code> (or <code>~/.clinerules</code>); copies skills alongside
x	codex-cli	writes <code>~/.codex/instructions.md</code> ; copies skills into <code>~/.codex/sauron-skills/</code>
a	aider	writes <code>./.aider.sauron.md</code> ; add to <code>.aider.conf.yml</code> as <code>read:</code>
g	generic	writes <code>./SYSTEM_PROMPT.sauron.md</code> you paste into any tool; copies skills alongside

2. The terminal session (Claude Code path)

```
.( 0 ).
```



interactive setup . one agent to route them all

0. Which agent orchestrator do you use?

- c) claude-code (default) full: hooks + Skill() invocations + delegate policy
 - r) cursor writes .cursor/rules/Framework.mdc (alwaysApply)
 - l) cline writes .clinerules with the delegate policy
 - x) codex-cli (OpenAI) writes ~/.codex/instructions.md
 - a) aider writes .aider.conf.yml + Framework.md conventions
 - g) generic / other writes SYSTEM_PROMPT.md you paste into any tool
- ✓ orchestrator: c

1. Where should it install?

- g) global → ~/.claude/settings.json (loads every session, every project)
- p) per-project → \$PWD/.claude/settings.json (only loads when Claude Code runs here)
- s) skip → print the settings JSON to stdout, do not write anything

2. Which skills should auto-load at session start?

These are invoked as the very first tool calls of every session.
Leave blank to skip a skill; type y to include it.

3. Which PAL models should be listed in the delegate-first policy?

Pick every model your PAL install actually has keys for.
Unselected models are dropped from the routing map so Claude never tries them.

Ready to write /tmp/sauron-pdf-v2/.claude/settings.json

Existing file will be backed up with .bak.<timestamp> suffix.

- ✓ wrote /tmp/sauron-pdf-v2/.claude/settings.json
- ✓ symlinked babysit-pr -> /tmp/sauron-pdf-v2/.claude/skills/babysit-pr
- ✓ symlinked debate-review -> /tmp/sauron-pdf-v2/.claude/skills/debate-review
- ✓ symlinked pal-router -> /tmp/sauron-pdf-v2/.claude/skills/pal-router
- ✓ symlinked validator -> /tmp/sauron-pdf-v2/.claude/skills/validator
- ✓ PAL MCP server is registered in ~/.claude.json
- ✓ wrote env template: /tmp/sauron-pdf-v2/.claude/.env.sauron.example

Next steps

1. Register PAL as an MCP server in ~/.claude.json:

```
"mcpServers": { "pal": { "type": "stdio", "command": "/path/to/zen-mcp-server/.pal_venv/bin/python", "args": ["/path/to/zen-mcp-server/server.py"], "env": { ...keys... } } }
```
 2. Source your API keys: `source /tmp/sauron-pdf-v2/.claude/.env.sauron.example` (after editing it)
 3. Restart Claude Code.
 4. On the next session start you should see the auto-invoked skills fire immediately.
- ✓ sauron setup complete.

3. What Claude sees at the next session start

The SessionStart hook fires *before* Claude responds. Its payload is injected into context.

SessionStart · additionalContext

MANDATORY FIRST ACTIONS: on session start and for EVERY model, BEFORE responding, immediately call the Skill tool for `caveman` (full mode), `pentesting-agent`, `validator` (preload). Full preference set: (1) delegate-first via PAL: groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge. (2) KEEP IN CLAUDE: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, tool-sequence orchestration. (3) On every confirmed finding run the 4-step pipeline: validator skill, gap tests, adversarial challenge via mcp_pal_challenge (pro fallback groq), synthesize; delegate report writing to groq.

UserPromptSubmit · additionalContext (every message)

Reminder every message: delegate aggressively to PAL to conserve tokens (groq (gpt-oss-120b, ~500 rpm, 8000 tpm cap) for report writing, vuln explanations, and skeptical validation. grok (x-ai, 2M ctx) for permissive high-context security reasoning when Gemini refuses. flash (gemini-3.6-flash, 1M ctx) for fast structured extraction. or-free (OpenRouter meta-router, 200K ctx) as generalist fallback. pro (gemini-3-pro-preview) for deep-reasoning fallback and adversarial debate via mcp_pal_challenge.). Keep in Claude ONLY: user-facing decisions, exploitation choices, severity calls, safety-boundary checks, side-effecting actions, orchestration. FAILOVER: if any model refuses or errors, immediately re-route; a refusal is a routing problem, not a stop.

For non-Claude orchestrators: the same doctrine gets written into that tool's rules file (e.g. Cursor's .mdc, Cline's .clinerules). Since only Claude Code has a Skill() primitive, non-Claude installs also append a skills index to the rules file so the model knows what SKILL.md files it can read when a trigger phrase appears.

4. Files on disk after install

```
/PROJECT/.claude/.env.sauron.example  
/PROJECT/.claude/settings.json  
/PROJECT/tree.txt
```

.env.sauron.example (only the API keys your model picks need)

```
# sauron API keys - source this from your shell rc, or export before starting Claude Code.  
# Do NOT commit the real values.  
export GEMINI_API_KEY=your-gemini-key  
export OPENROUTER_API_KEY=your-openrouter-key  
export CUSTOM_API_URL=https://api.groq.com/openai/v1  
export CUSTOM_API_KEY=your-groq-key
```

Repo: github.com/crowx01/sauron · Sibling: [crowx01/let-there-be-light](https://github.com/crowx01/let-there-be-light) (dev-lifecycle)